Lab 21

Collaborative AI Coding:

Pair Programming and Version Control Integration

Assignment Document

Lab Objectives

This lab covers the following objectives:

- Learn to use AI as a pair programmer for writing, reviewing, and optimizing code.
- Understand how to integrate AI-generated code into version control systems (Git).
- Practice collaborative workflows such as commit messages, branching, and merging.

Task 1 – AI-Assisted Pair Programming

Overview

In this task, we use AI to generate a To-Do List Manager application and then request AI-driven enhancements such as error handling, additional features, and code refactoring. The goal is to observe the collaborative role AI plays in iterative software development.

Step 1: Initial Program Generated by AI

Prompt given to AI:

"Generate a simple To-Do List Manager in Python with add, view, and delete task features."

```
# todo_manager.py - Initial Version

tasks = []

def add_task(task):
    """Add a new task to the list."""
    tasks.append(task)
    print(f"Task added: {task}")

def view_tasks():
    """Display all current tasks."""
    if not tasks:
        print("No tasks available.")
    else:
        for i, task in enumerate(tasks, 1):
            print(f"{i}. {task}")

def delete_task(task_number):
    """Remove a task by its number."""
    if 1 <= task_number <= len(tasks):
        removed = tasks.pop(task_number - 1)
        print(f"Removed task: {removed}")
    else:
        print("Invalid task number.")

# Main loop
while True:
    print("\n1. Add Task  2. View Tasks  3. Delete Task  4. Exit")
    choice = input("Choose an option: ")
    if choice == "1":
        add_task(input("Enter task: "))
    elif choice == "2":
        view_tasks()
    elif choice == "3":
        delete_task(int(input("Enter task number: ")))
    elif choice == "4":
        break
```

Step 2: AI-Suggested Enhancements

Prompt given to AI:

"Enhance the To-Do List Manager with error handling, task priorities, saving to a file, and input validation."

```
# todo_manager_enhanced.py - Improved Version

import json, os

TASK_FILE = 'tasks.json'

def load_tasks():
    """Load tasks from JSON file. Returns empty list if file absent."""
    if os.path.exists(TASK_FILE):
        with open(TASK_FILE, 'r') as f:
            return json.load(f)
    return []

def save_tasks(tasks):
    """Persist current task list to JSON file."""
    with open(TASK_FILE, 'w') as f:
        json.dump(tasks, f, indent=2)

def add_task(tasks, task, priority='normal'):
    """Add a task with optional priority (low/normal/high)."""
    if priority not in ('low', 'normal', 'high'):
        raise ValueError('Priority must be low, normal, or high.')
    tasks.append({'task': task, 'priority': priority, 'done': False})
    save_tasks(tasks)
    print(f"[{priority.upper()}] Task added: {task}")

def view_tasks(tasks):
    """Display all tasks with index, priority, and completion status."""
    if not tasks:
        print("No tasks found.")
        return
    for i, t in enumerate(tasks, 1):
        status = '[DONE]' if t['done'] else '[ ]'
        print(f"{i}. {status} [{t['priority'].upper()}] {t['task']}")

def delete_task(tasks, index):
    """Delete task by 1-based index with bounds validation."""
    if not (1 <= index <= len(tasks)):
        raise IndexError(f'No task at position {index}.')
    removed = tasks.pop(index - 1)
    save_tasks(tasks)
    print(f"Removed: {removed['task']}")

def mark_done(tasks, index):
    """Mark a task as completed by index."""
    if not (1 <= index <= len(tasks)):
        raise IndexError(f'No task at position {index}.')
    tasks[index - 1]['done'] = True
    save_tasks(tasks)
    print('Task marked as done.')

def main():
    tasks = load_tasks()
    while True:
```

```
        print('\n1.Add  2.View  3.Delete  4.Done  5.Exit')
        choice = input('Choose: ').strip()
        try:
            if choice == '1':
                t = input('Task: ').strip()
                p = input('Priority (low/normal/high): ').strip()
                add_task(tasks, t, p)
            elif choice == '2':
                view_tasks(tasks)
            elif choice == '3':
                delete_task(tasks, int(input('Task #: ')))
            elif choice == '4':
                mark_done(tasks, int(input('Task #: ')))
            elif choice == '5':
                break
        except (ValueError, IndexError) as e:
            print(f'Error: {e}')

if __name__ == '__main__':
    main()
```

Comparison: Original vs. Enhanced Version

Feature	Original Version	Enhanced Version
Error Handling	None	try/except with ValueError, IndexError
Task Priority	Not supported	Low / Normal / High levels
Persistence	In-memory only	Saved to tasks.json
Mark as Done	Not available	Supported with status flag
Input Validation	Minimal	Strict validation with feedback
Code Structure	Flat script	Functions + main() entry point

Task 2 – Version Control Integration

Overview

Version control with Git ensures that AI-assisted changes are tracked, reversible, and collaborative. Each AI-generated modification is committed to a dedicated branch and later merged into main.

Git Workflow Steps

Step 1: Initialize Repository

```
$ mkdir todo-app && cd todo-app
$ git init
Initialized empty Git repository in /todo-app/.git/
$ cp ~/todo_manager.py .
$ git add todo_manager.py
$ git commit -m "Initial commit: basic To-Do List Manager"
```

Step 2: Create Feature Branch

```
$ git checkout -b feature/ai-enhancements
Switched to a new branch 'feature/ai-enhancements'
```

Step 3: Add AI-Assisted Modifications

```
# Replace original file with enhanced version
$ cp ~/todo_manager_enhanced.py todo_manager.py
$ git add todo_manager.py
$ git commit -m "feat: add error handling, priority, persistence via AI assistance"
```

Step 4: Merge Back into Main

```
$ git checkout main
$ git merge feature/ai-enhancements
Updating 3a1b2c4..7f8e9d0
Fast-forward
 todo_manager.py | 48 ++++++++++++++++++++++++++++++++++++++++++++++++--------
 1 file changed, 40 insertions(+), 8 deletions(-)
```

Step 5: View Git Log

```
$ git log --oneline
7f8e9d0 feat: add error handling, priority, persistence via AI assistance
3a1b2c4 Initial commit: basic To-Do List Manager
```

Reflection: How Version Control Supports AI Collaboration

Version control plays a critical role when working with AI-generated code:

- Traceability: Every AI-suggested change is documented in the commit history with context.

- Reversibility: If an AI enhancement introduces bugs, git revert or git checkout restores the previous state instantly.
- Branching: AI experiments happen in isolated branches, keeping the main branch stable.
- Review: Team members can inspect diffs (git diff) to understand what the AI changed before merging.
- Accountability: Commit messages explain why each AI-driven change was made.

Task 3 – AI for Commit Message Generation

Overview

Meaningful commit messages are essential for maintainable Git history. In this task, we compare AI-generated commit messages against manually written ones and evaluate their usefulness.

Changes Made

- Added task priority system (low/normal/high).
- Implemented JSON-based file persistence for tasks.
- Added input validation and error handling using try/except.
- Added mark_done() function to track task completion.

Prompt to AI for Commit Message

"Generate a concise, meaningful Git commit message for the following changes: added task priority levels, JSON file persistence, error handling with try/except, and a mark_done() function."

Comparison: AI vs Human Commit Messages

Aspect	Human-Written Message	AI-Generated Message
Message Text	"updated todo app"	"feat: add priority levels, persistence, error handling & done tracking"
Clarity	Vague	Clear and specific
Completeness	Mentions no features	Lists all major changes
Convention	No prefix used	Follows Conventional Commits format
Length	3 words	10–12 words, well-structured
Traceability	Hard to trace changes	Easy to understand intent from history

AI-Generated Commit Messages (Samples)

```
# Short commit message (AI-generated):
feat: add task priorities, JSON persistence, error handling, done flag

# Detailed commit message (AI-generated):
feat(todo): enhance task manager with priorities and persistence

- Add low/normal/high priority levels to tasks
- Persist tasks to tasks.json using json module
- Handle ValueError and IndexError in user input
- Add mark_done() to flag tasks as completed
```

```
Co-authored-with: Claude AI <ai@anthropic.com>
```

Discussion

AI-generated commit messages significantly improve documentation quality in Git history:

- They follow community conventions like Conventional Commits (feat:, fix:, chore:).
- They describe what changed AND why, making logs self-documenting.
- They reduce the cognitive load on developers who struggle to write descriptive messages quickly.
- When given enough context, AI messages are more complete and specific than the average human message written under time pressure.

Task 4 – Pair Programming Simulation

Overview

In this simulation, Student A writes the initial version of a program and Student B, assisted by AI, improves it and adds features. Both collaborate through Git branching and merging to integrate their work.

Student A – Initial Program

Student A writes a basic Student Grade Tracker:

```
# grade_tracker.py - Student A's initial version

students = {}

def add_grade(name, grade):
    students[name] = grade
    print(f'Added grade for {name}: {grade}')

def show_grades():
    for name, grade in students.items():
        print(f'{name}: {grade}')

add_grade('Alice', 85)
add_grade('Bob', 72)
show_grades()
```

Student A – Git Steps

```
$ git init grade-tracker && cd grade-tracker
$ git add grade_tracker.py
$ git commit -m "feat: initial grade tracker by Student A"
```

Student B – AI-Assisted Improvements

Prompt given to AI:

"Improve this grade tracker to support multiple subjects per student, calculate averages, assign letter grades, and handle invalid input."

```
# grade_tracker_enhanced.py - Student B with AI assistance

students = {}

def add_grade(name, subject, score):
    """Add a score for a student in a specific subject."""
    if not (0 <= score <= 100):
        raise ValueError(f'Score must be 0-100. Got: {score}')
    students.setdefault(name, {})[subject] = score
    print(f'[{name}] {subject}: {score}')
```

```
def get_average(name):
    """Return the average score for a student across all subjects."""
    grades = students.get(name, {}).values()
    return round(sum(grades) / len(grades), 2) if grades else 0

def get_letter(avg):
    """Convert numeric average to a letter grade."""
    if avg >= 90: return 'A'
    elif avg >= 80: return 'B'
    elif avg >= 70: return 'C'
    elif avg >= 60: return 'D'
    return 'F'

def show_report():
    """Print a formatted report card for all students."""
    print('\n--- Grade Report ---')
    for name in students:
        avg = get_average(name)
        letter = get_letter(avg)
        print(f'{name}: Avg={avg}, Grade={letter}')
        for subj, score in students[name].items():
            print(f'  {subj}: {score}')

try:
    add_grade('Alice', 'Math', 92)
    add_grade('Alice', 'Science', 88)
    add_grade('Bob', 'Math', 70)
    add_grade('Bob', 'Science', 65)
    show_report()
except ValueError as e:
    print(f'Error: {e}')
```

Student B – Git Steps

```
$ git checkout -b feature/student-b-improvements
$ cp grade_tracker_enhanced.py grade_tracker.py
$ git add grade_tracker.py
$ git commit -m "feat: multi-subject grades, averages, letter grades (AI-assisted)"
```

Merging Both Students' Work

```
$ git checkout main
$ git merge feature/student-b-improvements
Merging feature/student-b-improvements into main...
Fast-forward
 grade_tracker.py | 52 ++++++++++++++++++++++++++++++++++---
1 file changed, 48 insertions(+), 4 deletions(-)
```

Final Comparison: Before vs After

Feature	Student A (Initial)	Student B + AI
Data Structure	name → single grade	name → dict of subjects
Multiple Subjects	Not supported	Fully supported

Average Calculation	None	Auto-calculated per student
Letter Grade	None	A/B/C/D/F assignment
Input Validation	None	Score range 0–100 enforced
Report Output	Simple print	Formatted report card
Error Handling	None	try/except ValueError

Reflection on Pair Programming with AI

The pair programming simulation demonstrated several key insights:

- AI accelerates the improvement cycle: Student B was able to add complex features quickly by leveraging AI suggestions.
- Branching preserves Student A's work: The original version remains intact in the main branch history.
- Git merging enables seamless integration: Both contributions are preserved and traceable in the commit log.
- AI as a silent third partner: AI acts as an always-available collaborator who knows best practices and can generate boilerplate instantly.
- Human oversight remains essential: Student B reviewed, tested, and refined the AI output before committing.

Summary

This lab demonstrated how AI can be effectively integrated into the software development workflow as a collaborative coding partner. The following outcomes were achieved:

Task	Activity	Key Outcome
Task 1	AI Pair Programming	Generated and enhanced a To-Do Manager with AI
Task 2	Version Control Integration	Used Git branches and merges with AI-assisted code
Task 3	AI Commit Messages	AI produces more specific and conventional messages
Task 4	Pair Programming Simulation	Students collaborated via Git with AI as a co-developer

AI tools are most effective when combined with human judgment, version control discipline, and clear communication. The ability to branch, commit, and merge ensures that AI-generated code remains auditable, reversible, and maintainable over time.